

ODOO BACKEND DEVELOPMENT ESSENTIALS

Odoo Core Development

Install, modules, fields, and ORM basics

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: Odoo 17 Install Overview to Hide the Database List — 7 topics
- 2 Section 2: Understand the Master Password to Recognize Module Folder Structure — 7 topics
- 3 Section 3: Define a Model with `_name` to Use an Integer Field — 7 topics
- 4 Section 4: Use a Float Field to Create a Compute Field — 7 topics
- 5 Section 5: Create a Related Field to Count Records with `search_count` — 7 topics
- 6 Section 6: Use `browse()` and `exists()` to Use `sudo` with Caution — 7 topics

7 Section 7: Create Related Record with [0, 0, vals] to Introduce CSV Access Rights — 7 topics

1 Section 1

Odoo 17 Install Overview to Hide the Database List

1.1 Odoo 17 Install Overview

- Odoo needs Python, PostgreSQL, and system libraries.
- A source install is useful for training.

```
user@odoo-lab:~/project

trainee@odoo-lab:/opt/odoo$ ./odoo-bin shell -d mydb
>>> env['res.partner'].search_count({})
42
```

1.2 Start Odoo with odoo-bin

- odoo-bin starts the Odoo server.
- Run it from the source checkout.

```
user@odoo-lab:~/project

trainee@odoo-lab:/opt/odoo$ ./odoo-bin shell -d mydb
>>> env['res.partner'].search_count({})
42
```

1.3 Understand odoo.conf Basics

- The config file stores common startup options.
- It keeps long commands readable.

```
user@odoo-lab:~/project

trainee@odoo-lab:/opt/odoo$ ./odoo-bin shell -d mydb
>>> env['res.partner'].search_count({})
42
```

1.4 Set db_host, db_user, and db_password

- These options tell Odoo how to reach PostgreSQL.

Config file values

```
db_host = 127.0.0.1  
db_user = odoo17  
db_password = change-me
```

- Use a dedicated PostgreSQL role.

CLI values

```
./odoo-bin --db_host=127.0.0.1 --db_user=odoo17 --db_password=change-me
```


1.5 Configure addons_path

- addons_path tells Odoo where modules live.

Config addons path

```
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/custom-addons
```

- Separate core and custom addons clearly.

CLI addons path

```
./odoo-bin --addons-path=addons,../custom-addons -d training
```

1.6 Write Logs to a File

- Logs show startup and runtime problems.

Config logfile

```
logfile = /var/log/odoo/odoo17.log
```

- A logfile is easier to inspect later.

View recent logs

```
less /var/log/odoo/odoo17.log
```

1.7 Hide the Database List

- list_db controls the web database selector.

Disable listing

```
list_db = False
```

- Set it false on shared or public servers.

Start with one database

```
./odoo-bin -c odoo17.conf -d odoo17_prod
```

2 Section 2

Understand the Master Password to Recognize Module Folder Structure

2.1 Understand the Master Password

- The master password protects database manager actions.

Config setting

```
admin_passwd = replace-with-a-strong-secret
```

- Use a strong value outside source control.

Database manager use

```
Open /web/database/manager  
Enter the master password for backup or restore actions
```

2.2 Install a Module with -i

- -i installs a module into a database.

Install base CRM

```
./odoo-bin -d training -i crm --stop-after-init
```

- Use it when the module is not installed yet.

Install custom module

```
./odoo-bin -c odoo17.conf -d training -i estate --stop-after-init
```

2.3 Upgrade a Module with -u

- -u updates an installed module.

Upgrade one module

```
./odoo-bin -d training -u estate --stop-after-init
```

- Use it after changing models, views, or data.

Upgrade several modules

```
./odoo-bin -c odoo17.conf -d training -u estate,sale --stop-after-init
```

2.4 Use shell -d

- The Odoo shell opens an ORM-aware Python prompt.

Open shell

```
./odoo-bin shell -d training
```

- Use -d to choose the database.

Inspect a model

```
env['res.partner'].search([], limit=3)
```


2.5 Scaffold a New Module

- Scaffold creates a starter module directory.

Create module skeleton

```
./odoo-bin scaffold estate ../custom-addons
```

- It gives common files and directories.

Generated path

```
../custom-addons/estate/__manifest__.py  
../custom-addons/estate/models/models.py
```

2.6 Set Manifest name, version, and depends

- The manifest describes the module.

Basic manifest

```
{  
    'name': 'Estate',  
    'version': '17.0.1.0.0',  
    # ...
```

- depends controls module load order.

Depend on sales

```
{  
    'name': 'Estate Sale',  
    'depends': ['estate', 'sale'],  
    # ...
```

2.7 Recognize Module Directory Structure

- Python models usually live in models.

Common folders

```
estate/  
  __manifest__.py  
  models/  
# ...
```

- XML views usually live in views.

Import models

```
# estate/models/__init__.py  
from . import estate_property
```

3 Section 3

Define a Model with `_name` to Use an Integer Field

3.1 Define a Model with `_name`

- `_name` gives the model its technical name.

New model

```
class EstateProperty(models.Model):  
    _name = 'estate.property'
```

- Use dotted names such as `estate.property`.

Use model name

```
self.env['estate.property'].search([])
```

3.2 Add a Model _description

- _description gives a readable model label.

Description line

```
class EstateProperty(models.Model):  
    _name = 'estate.property'  
    _description = 'Estate Property'
```

- It helps logs and technical screens.

Another description

```
class EstateOffer(models.Model):  
    _name = 'estate.offer'  
    _description = 'Estate Offer'
```

3.3 Use a Char Field

- Char stores short text.

Name field

```
name = fields.Char(string='Name', required=True)
```

- Use it for names, codes, and labels.

Reference field

```
reference = fields.Char(string='Reference', copy=False)
```

3.4 Use a Text Field

- Text stores longer text.

Description field

```
description = fields.Text(string='Description')
```

- Use it for notes and descriptions.

Internal notes

```
note = fields.Text(string='Internal Notes')
```


3.5 Use a Boolean Field

- Boolean stores true or false.

Active flag

```
is_published = fields.Boolean(string='Published')
```

- Use it for flags and toggles.

Default true

```
active = fields.Boolean(default=True)
```

3.6 Use a Selection Field

- Selection stores one value from a fixed list.

State field

```
state = fields.Selection([
    ('new', 'New'),
    ('done', 'Done'),
    # ...
```

- Each option has a key and label.

Priority field

```
priority = fields.Selection([
    ('0', 'Normal'),
    ('1', 'High'),
    # ...
```

3.7 Use an Integer Field

- Integer stores whole numbers.

Quantity field

```
quantity = fields.Integer(string='Quantity')
```

- Use it for counts and quantities.

Bedrooms field

```
bedrooms = fields.Integer(default=2)
```

4

Section 4

Use a Float Field to Create a Compute Field

4.1 Use a Float Field

- Float stores decimal numbers.

Area field

```
living_area = fields.Float(string='Living Area')
```

- Use it for measurements and rates.

Percentage field

```
commission_rate = fields.Float(string='Commission Rate')
```

4.2 Use a Date Field

- Date stores a calendar day.

Simple date

```
available_date = fields.Date(string='Available From')
```

- Use it when time of day is not needed.

Default today

```
date_order = fields.Date(default=fields.Date.today)
```

4.3 Use a Datetime Field

- Datetime stores date and time.

Meeting time

```
meeting_at = fields.Datetime(string='Meeting Time')
```

- Use it for events and timestamps.

Default now

```
created_marker = fields.Datetime(default=fields.Datetime.now)
```

4.4 Use a Many2one Field

- Many2one links this record to one other record.

Partner link

```
partner_id = fields.Many2one('res.partner', string='Customer')
```

- It creates a database foreign key.

Required salesperson

```
user_id = fields.Many2one('res.users', required=True)
```


4.5 Use a One2many Field

- One2many shows records pointing back to this record.

Order lines

```
line_ids = fields.One2many('sale.order.line', 'order_id')
```

- It needs the child model and inverse field.

Estate offers

```
offer_ids = fields.One2many('estate.offer', 'property_id')
```

4.6 Use a Many2many Field

- Many2many links many records on both sides.

Tag field

```
tag_ids = fields.Many2many('estate.property.tag', string='Tags')
```

- Odoo stores links in a relation table.

Allowed users

```
allowed_user_ids = fields.Many2many('res.users')
```

4.7 Create a Compute Field

- A compute field gets its value from a method.

Compute total

```
total_area = fields.Float(compute='_compute_total_area')  
  
@api.depends('living_area', 'garden_area')  
# ...
```

- Use depends to declare trigger fields.

Stored compute

```
best_price = fields.Float(compute='_compute_best_price', store=True)
```

5 Section 5

Create a Related Field to Count Records with `search_count`

5.1 Create a Related Field

- A related field follows a relationship path.

Partner email

```
partner_email = fields.Char(related='partner_id.email')
```

- It avoids duplicating simple values.

Stored company

```
company_id = fields.Many2one(related='user_id.company_id', store=True)
```

5.2 Understand the create Override Idea

- create runs when new records are made.

Basic create override

```
@api.model_create_multi
def create(self, vals_list):
    records = super().create(vals_list)
    # ...
```

- Call super to keep standard behavior.

Fill a missing value

```
for vals in vals_list:
    vals.setdefault('state', 'new')
```

5.3 Understand the write Override Idea

- write runs when existing records change.

Basic write override

```
def write(self, vals):  
    result = super().write(vals)  
    return result
```

- Call super once for the normal update.

React to field change

```
if 'state' in vals:  
    self.message_post(body='State changed')
```

5.4 Understand the unlink Idea

- unlink runs when records are deleted.

Basic unlink override

```
def unlink(self):  
    return super().unlink()
```

- Use it to block unsafe deletion.

Block deletion

```
if any(record.state == 'done' for record in self):  
    raise UserError('Done records cannot be deleted.')
```


5.5 Find Records with search()

- search returns a recordset.

Find customers

```
customers = self.env['res.partner'].search([('customer_rank', '>', 0)])
```

- Pass a domain to filter records.

Find one record

```
partner = self.env['res.partner'].search([('email', '=', email)], limit=1)
```

5.6 Build a domain = Filter

- A domain is a list of filter conditions.

Simple domain

```
domain = [('state', '=', 'sale')]
```

- Each condition is field, operator, value.

Multiple conditions

```
domain = [('amount_total', '>=', 1000), ('partner_id', '!=', False)]
```

5.7 Count Records with search_count

- search_count returns an integer.

Count customers

```
count = self.env['res.partner'].search_count([('customer_rank', '>', 0)])
```

- It is faster than loading records just to count.

Count open tasks

```
open_tasks = self.env['project.task'].search_count([('stage_id.fold', '=', False)])
```

6 Section 6

Use browse() and exists() to Use sudo with Caution

6.1 Use browse() and exists()

- browse creates a recordset from ids.

Browse by id

```
partner = self.env['res.partner'].browse(42)
```

- It does not guarantee records exist.

Check existence

```
partner = self.env['res.partner'].browse(42).exists()
```

6.2 Use ensure_one

- ensure_one confirms the recordset has one record.

Single-record method

```
def action_confirm(self):  
    self.ensure_one()  
    self.state = 'confirmed'
```

- It raises an error for zero or many records.

Read one value

```
self.ensure_one()  
return self.name
```

6.3 Use mapped

- mapped extracts a field from each record.

Names list

```
names = partners.mapped('name')
```

- It can follow relational paths.

Related records

```
companies = partners.mapped('company_id')
```

6.4 Use filtered

- filtered keeps records matching Python logic.

Keep active records

```
active_partners = partners.filtered(lambda p: p.active)
```

- Use search domains when the database can filter.

Keep high value orders

```
large_orders = orders.filtered(lambda o: o.amount_total > 1000)
```


6.5 Use active for Archive Behavior

- The active field controls archive behavior.

Add active field

```
active = fields.Boolean(default=True)
```

- Archived records are hidden from normal searches.

Archive records

```
records.action_archive()  
records.action_unarchive()
```

6.6 Read the Current User with env.user

- env.user is the current Odoo user.

Default salesperson

```
user_id = fields.Many2one('res.users', default=lambda self: self.env.user)
```

- Use it for defaults and simple checks.

Check groups

```
if self.env.user.has_group('sales_team.group_sale_manager'):  
    return True
```

6.7 Use sudo with Caution

- sudo bypasses normal access checks.

Controlled sudo read

```
company = self.env['res.company'].sudo().browse(company_id)
```

- Use it only when the business case is clear.

Prefer normal access

```
partners = self.env['res.partner'].search(domain)
```

7 Section 7

Create Related Record with [0, 0, vals] to Introduce CSV Access Rights

7.1 Create Related Record with [0, 0, vals]

- [0, 0, vals] creates a new related record.

Create order line

```
order.write({'order_line': [(0, 0, {'name': 'Service', 'price_unit': 100})]})
```

- Use it in One2many or Many2many updates.

Create tag

```
record.write({'tag_ids': [(0, 0, {'name': 'New Tag'})]})
```

7.2 Update Related Record with [1, id, vals]

- [1, id, vals] updates an existing related record.

Update line price

```
order.write({'order_line': [(1, line_id, {'price_unit': 120})]})
```

- The id is the related record id.

Update tag name

```
record.write({'tag_ids': [(1, tag_id, {'name': 'Updated'})]})
```

7.3 Delete Related Record with [2, id]

- [2, id] removes and deletes the related record.

Delete order line

```
order.write({'order_line': [(2, line_id)]})
```

- Use it carefully with business data.

Delete related tag

```
record.write({'tag_ids': [(2, tag_id)]})
```

7.4 Link Existing Record with [4, id]

- [4, id] links an existing record.

Add existing tag

```
record.write({'tag_ids': [(4, tag_id)]})
```

- It does not create or delete the linked record.

Add existing follower group

```
record.write({'allowed_group_ids': [(4, group_id)]})
```


7.5 Replace Links with [6, 0, ids]

- [6, 0, ids] replaces all current links.

Replace tags

```
record.write({'tag_ids': [(6, 0, [1, 2, 3])])})
```

- ids is the full final list.

Use ids from records

```
record.write({'user_ids': [(6, 0, users.ids)])})
```

7.6 Create an XML Data Record

- XML data creates or updates records during module load.

Simple XML record

```
<record id="estate_tag_garden" model="estate.property.tag">
  <field name="name">Garden</field>
</record>
```

- The id becomes an external identifier.

Manifest data entry

```
'data': [
    'views/estate_property_views.xml',
    'data/estate_tags.xml',
    # ...
```

7.7 Introduce CSV Access Rights

- `ir.model.access.csv` grants model permissions.

CSV header

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
```

- Access rights cover read, write, create, and delete.

Basic access row

```
access_estate_property_user,estate.property.user,model_estate_property,base.group_user,1,1,1,0
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory

04 · Odoo Core

